CATEGORY B

Part A — Coding Problems

Problem 1: Library Catalog Lookup

TASK

A library maintains a catalog of books as a list of records. Each record contains an ISBN (a unique numeric string) and a title. The catalog is pre-sorted by ISBN in ascending order. You need to implement a function to efficiently find a book's title given its ISBN.

Implement a function `findBook(catalog, targetIsbn)` that takes a sorted list of `(ISBN, title)` tuples and a `targetIsbn` string. It should return the title if the `targetIsbn` is found, or the string "Not Found" otherwise.

Your solution must leverage the fact that the catalog is sorted to achieve optimal performance for lookup operations.

EXAMPLES

Example 1

```
catalog = [("0001112223", "Introduction to Algebra"),
           ("0002223334", "Beginning Python"),
           ("0003334445", "Classic Mythology"),
           ("0004445556", "Data and Society"),
           ("0005556667", "European History")]
targetIsbn = "0003334445"

Expected Output: "Classic Mythology"
```

Example 2

```
targetIsbn = "0009998887"

Expected Output: "Not Found"
```

COMPLEXITY REQUIREMENTS

State the time and additional space complexity of your solution.

HINT

Consider algorithms that are efficient for searching in sorted lists. How can you repeatedly reduce the search space? Think about binary search.

Problem 2: Warehouse Grid Summary

TASK

Implement a function `warehouseSummary(grid)` that processes a 2D grid representing a warehouse floor layout. Each cell `(row, col)` in the grid contains a non-negative integer representing the count of items in a storage bin.

The function should calculate the total number of items across all bins and identify the coordinates `(row, col)` of the bin holding the maximum number of items. If there are multiple bins with the same maximum item count, return the coordinates of the first one encountered when scanning row by row, from left to right.

INPUT / OUTPUT

Input: `grid` — a list of lists of integers, representing the 2D grid.

Output: A tuple `(total_items, max_coordinate)` where `total_items` is the sum of all items, and `max_coordinate` is a tuple `(row, col)` indicating the position of the bin with the most items.

EXAMPLE

Example

```
grid = [[4, 9, 2],
        [7, 1, 6],
        [3, 12, 5]]

Expected Output: (49, (2, 1))
```

COMPLEXITY REQUIREMENTS

State the time complexity in terms of m (rows) and n (columns) and the additional space complexity beyond the input grid itself.

HINT

You will need to iterate through all elements of the 2D array. Keep track of the running total and the maximum value found along with its coordinates.

Problem 3: Pair With Target Sum (Unsorted)

TASK

Given a list of integers and a target value, determine if there are any two distinct elements in the list that add up to the target.

Implement a function `hasPairWithSum(nums, target)` that takes an array of integers `nums` (which may include negatives and duplicates, and is in no particular order) and an integer `target`. The function should return `true` if any two distinct elements in `nums` sum up to `target`, and `false` otherwise.

EXAMPLES

Example 1

```
nums = [2, 7, 11, 15], target = 9

Expected Output: true (because 2 + 7 = 9)
```

Example 2

```
nums = [3, 4, 6], target = 20

Expected Output: false
```

COMPLEXITY REQUIREMENTS

Discuss at least two approaches to solve this problem, highlighting their different time and space complexity trade-offs. State the complexity of each approach.

HINT

For an optimal solution, consider how you can efficiently check for the 'complement' of each number. A brute-force approach would involve checking every possible pair.

Problem 4: Pair With Target Sum (Unsorted Array)

TASK

Given an array of integers and a target integer, determine if there exist any two distinct elements in the array that sum up to the target value.

INPUT / OUTPUT

Input: `nums` — an array of integers (can contain negatives and duplicates), e.g., `[2, 7, 11, 15]`; `target` — an integer, e.g., `9`.

Output: `true` if two distinct elements sum to the target, `false` otherwise.

EXAMPLES

Sample 1

```
nums = [2, 7, 11, 15], target = 9

Expected Output: true (because 2 + 7 = 9)
```

Sample 2

```
nums = [3, 4, 6], target = 20

Expected Output: false
```

COMPLEXITY REQUIREMENTS

Discuss at least two approaches to solve this problem, highlighting their different time/space trade-offs. State the time and space complexity for each approach.

HINT

One approach could involve iterating through all possible pairs. Another might use a data structure to quickly check for complements.

Problem 5: Maximize Area Between Two Boundaries

TASK

Imagine an array of non-negative integers where each element `heights[i]` represents the height of a vertical line at position `i`. These lines are drawn such that their base is on the x-axis. You need to find two lines that, along with the x-axis, form a container that can hold the most water.

Implement a function `maxContainerArea(heights)` that takes an array of non-negative integers `heights`. It should return the maximum amount of water that can be contained. The area between two walls `i` and `j` is calculated as `min(heights[i], heights[j]) * (j - i)`.

EXAMPLE

Example

```
heights = [1, 8, 6, 2, 5, 4, 8, 3, 7]

Expected Output: 49

// Lines at index 1 (height 8) and index 8 (height 7):
// min(8, 7) * (8 - 1) = 7 * 7 = 49
```

COMPLEXITY REQUIREMENTS

Describe the time complexity of a brute-force solution that checks every pair of walls, and then state the time and additional space complexity of a more optimal solution. Explain why the optimal solution is preferable for very large inputs.

HINT

A brute-force solution would involve nested loops. For a more optimal approach, consider starting with two pointers at the extreme ends and moving them inward. How does moving a pointer based on the shorter wall help optimize the search?

Part B — Quiz

Choose the single best answer for each question.

Q1. What is the time complexity for accessing an element at a known index in a one-dimensional (1D) array?

A. O(n)
B. O(1)
C. O(log n)
D. $O(n^2)$

Q2. The operation of adding a new element to a data structure is known as what?

A. Traversal
B. Deletion
C. Insertion
D. Searching

Q3. When choosing an appropriate data structure for a given problem, what is the most crucial step?

A. Always selecting an array because it's simple
B. Ignoring memory constraints to optimize for speed
C. Matching the required operations and constraints to the strengths of different data structures
D. Using only primitive data types to avoid complexity

Q4. **A software system stores frequently accessed calculations in a cache (extra memory) to avoid recomputing them, thereby speeding up future queries. This scenario best illustrates what concept?**

A. Primitive vs non-primitive data distinction
B. Linear vs non-linear data structures
C. Time vs space trade-offs
D. Big-O notation simplification

Q5. **A function has two distinct loops. The first loop iterates m times over one input, and the second loop (not nested) iterates n times over a different input. What is the overall time complexity?**

A. $O(m \times n)$
B. $O(m + n)$
C. $O(m^2)$
D. $O(n^2)$

Q6. **In which type of data structure are elements arranged sequentially, where each element (except the first/last) has exactly one predecessor and one successor?**

A. Non-linear data structure
B. Hierarchical data structure
C. Linear data structure
D. Networked data structure

Q7. **An algorithm performs a number of operations proportional to 2n + 5. According to the rules of Big-O notation, what is its simplified time complexity?**

A. $O(2n)$
B. $O(n + 5)$
C. $O(n)$
D. $O(1)$

Q8. **Which of the following is an example of a non-primitive data structure?**

A. int
B. char
C. boolean
D. Linked list

Q9. **For a two-dimensional (2D) array with m rows and n columns, what is the time complexity of visiting every element exactly once?**

A. $O(m + n)$
B. $O(m \times n)$
C. $O(\log(m \times n))$
D. $O(1)$

Q10. **What is the fundamental purpose of a data structure?**

A. To make a program incorrect or slow

B. To define basic variables in a language

C. To organize, store, and access data efficiently for specific operations

D. To exclusively manage memory allocation

Part C — Concept Questions

Answer each question in a few sentences, giving reasons for your answer.

Question 1

Compare the time complexity for searching for a specific element in a 1D array when the array is unsorted versus when it is sorted. Explain the fundamental reason for this difference in search efficiency.

Question 2

When determining the Big-O complexity of an algorithm, constant multipliers are dropped. Explain why an algorithm performing '2n' operations is still classified as O(n), rather than O(2n), in Big-O notation.

Question 3

Describe how the traversal strategy fundamentally differs between linear data structures (like arrays) and non-linear data structures (like trees), based on how elements are related.